Lab 03 — Deception: Honeypots, Honeyfiles and Honeytokens

> **SY0-701 Objective 1.2** — Summarize fundamental security concepts.
>
> **Domain 1** — General Security Concepts (12%) · **Time:** ~35 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Deploy four kinds of deception technology on one host and prove that each one raises an alert the moment an "attacker" touches it.

You'll build: A low-interaction honeypot that logs every connection, a honeyfile watched by auditd and inotify, a honeytoken (decoy cloud credential) with a detection script, a two-decoy honeynet inside a network namespace, and a consolidated alert summary.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 01 (auditd basics) is helpful; tools installed in step 1.

> **Legal note:** Deploy decoys only on systems you own or are authorised to manage. All "attacker" traffic in this lab is generated by you against `127.0.0.1` and a private namespace inside the playground. Unauthorised access to computer material is an offence under Singapore's Computer Misuse Act 1993.

Background

- **Deception and disruption technology** turns the attacker's curiosity against them: decoys have no legitimate users, so *any* interaction is a high-fidelity signal with almost no false positives.
- A **honeypot** is a decoy system or service; a **honeynet** is a network of honeypots that imitates a realistic environment.
- A **honeyfile** is a decoy file with a tempting name; a **honeytoken** is decoy data such as a fake credential, API key or database record whose use proves a breach.
- Low-interaction honeypots only emulate a banner, which is safe and simple; high-interaction honeypots run real services and need strict isolation.
- Decoys are detective controls. They also disrupt attackers by wasting their time and revealing their tools and techniques.

Steps

1. Install the tools

auditd records who opened the honeyfile, inotify-tools gives a real-time alert, netcat plays the attacker, and iproute2 builds the honeynet namespace.

```
apt-get update -qq && apt-get install -y auditd inotify-tools netcat-openbsd iproute2
python3 curl
systemctl enable --now auditd
mkdir -p /opt/honeypot /root/lab03 && touch /var/log/honeypot.log
```

2. Write a low-interaction honeypot

The script listens on a port, presents a fake service banner, captures the first bytes the client sends, and writes one log line per connection. It never executes anything it receives.

```
cat > /opt/honeypot/honeypot.py <<'EOF'
#!/usr/bin/env python3
"""Low-interaction honeypot: honeypot.py PORT BANNER LOGFILE"""
import datetime, socketserver, sys

PORT, BANNER, LOG = int(sys.argv[1]), sys.argv[2].encode() + b"\r\n", sys.argv[3]

class Decoy(socketserver.BaseRequestHandler):
    def handle(self):
        self.request.settimeout(5)
        try:
            self.request.sendall(BANNER)
            data = self.request.recv(1024)
        except OSError:
            data = b""
        ts = datetime.datetime.now().isoformat(timespec="seconds")
        src = f"{self.client_address[0]}:{self.client_address[1]}"
        with open(LOG, "a") as f:
            f.write(f"{ts} HONEYPOT port={PORT} src={src} data={data[:60]!r}\n")

socketserver.ThreadingTCPServer.allow_reuse_address = True
socketserver.ThreadingTCPServer(("0.0.0.0", PORT), Decoy).serve_forever()
EOF
chmod 755 /opt/honeypot/honeypot.py && python3 -m py_compile /opt/honeypot/honeypot.py &&
echo "honeypot OK"
```

3. Deploy the honeypot and simulate an attacker probe

Run a fake SSH service on port 2222 that advertises an old, attractive OpenSSH version, then probe it the way an attacker would.

```
nohup python3 /opt/honeypot/honeypot.py 2222 "SSH-2.0-OpenSSH_7.4" /var/log/honeypot.log
>/dev/null 2>&1 &
sleep 1 && ss -tlnp | grep 2222
printf 'root:toor\n' | nc -w 3 127.0.0.1 2222
sleep 1 && tail -2 /var/log/honeypot.log
```

```
SSH-2.0-OpenSSH_7.4
2026-10-28T10:15:02 HONEYPOT port=2222 src=127.0.0.1:51234 data=b'root:toor\n'
```

4. Plant a honeyfile and watch it with auditd

A file called `payroll-2026-FINAL.csv` on a share is irresistible. Nobody legitimate needs it, so every read is suspicious. auditd records *who* opened it and with which program.

```
mkdir -p /srv/finance
cat > /srv/finance/payroll-2026-FINAL.csv <<'EOF'
employee_id,name,monthly_salary
E1001,DECOY RECORD,0
E1002,DECOY RECORD,0
EOF
chmod 644 /srv/finance/payroll-2026-FINAL.csv
auditctl -w /srv/finance/payroll-2026-FINAL.csv -p r -k honeyfile
id intruder >/dev/null 2>&1 || useradd -m intruder
sudo -u intruder cat /srv/finance/payroll-2026-FINAL.csv >/dev/null
sleep 1 && ausearch -k honeyfile -i | grep "type=SYSCALL" | tail -1
```

Look for `uid=intruder` and `exe=/usr/bin/cat` in the output.

5. Raise a real-time honeyfile alert with inotifywait

auditd is great evidence, but a SOC also wants an immediate alert. inotify fires the instant the file is opened.

```
nohup inotifywait -m -e open --timefmt '%F %T' --format '%T HONEYFILE %e %w%f'
/srv/finance/payroll-2026-FINAL.csv >> /var/log/honeyfile-alert.log 2>/dev/null &
sleep 1
sudo -u intruder head -1 /srv/finance/payroll-2026-FINAL.csv >/dev/null
sleep 1 && cat /var/log/honeyfile-alert.log
```

6. Plant a honeytoken and detect its use

Leave a fake cloud credential where an intruder will look. It grants nothing, so the only way it can appear in traffic is if someone stole it. A decoy API logs requests, and a detection script alerts on the token value.

```
mkdir -p /root/.aws
cat > /root/.aws/credentials.bak <<'EOF'
[prod-admin]
aws_access_key_id = AKIAHONEYTOKEN0LAB03
aws_secret_access_key = LabOnlyDecoySecretNotReal0000000000000000
EOF
mkdir -p /srv/decoy-api && echo "ok" > /srv/decoy-api/index.html
nohup python3 -m http.server 8088 --directory /srv/decoy-api >> /var/log/decoy-api.log
2>&1 &
sleep 1
curl -s -o /dev/null "http://127.0.0.1:8088/v1/login?access_key=AKIAHONEYTOKEN0LAB03"
cat > /usr/local/bin/honeytoken-check.sh <<'EOF'
#!/bin/bash
TOKEN="AKIAHONEYTOKEN0LAB03"
if grep -q "$TOKEN" /var/log/decoy-api.log; then
  echo "ALERT: honeytoken $TOKEN was used - treat /root/.aws as compromised"
  grep "$TOKEN" /var/log/decoy-api.log | tail -3
else
  echo "no honeytoken use detected"
fi
EOF
chmod 755 /usr/local/bin/honeytoken-check.sh && /usr/local/bin/honeytoken-check.sh
```

7. Build a mini honeynet in a network namespace

A honeynet looks like a small network of real hosts. Create an isolated namespace with two decoy IP addresses, run a fake FTP server and a fake Telnet server inside it, then scan and probe it from the host.

```
ip netns add honeynet 2>/dev/null || true
ip link add veth-host type veth peer name veth-hn 2>/dev/null && ip link set veth-hn netns
honeynet
ip addr add 10.66.0.1/24 dev veth-host 2>/dev/null; ip link set veth-host up
ip netns exec honeynet ip addr add 10.66.0.2/24 dev veth-hn 2>/dev/null
ip netns exec honeynet ip addr add 10.66.0.3/24 dev veth-hn 2>/dev/null
ip netns exec honeynet ip link set veth-hn up && ip netns exec honeynet ip link set lo up
ip netns exec honeynet nohup python3 /opt/honeypot/honeypot.py 21 "220 ProFTPD 1.3.5
Server (FILESRV01)" /var/log/honeypot.log >/dev/null 2>&1 &
ip netns exec honeynet nohup python3 /opt/honeypot/honeypot.py 23 "Ubuntu 14.04 LTS
PAYROLL-DB login:" /var/log/honeypot.log >/dev/null 2>&1 &
sleep 1
nc -zv -w 2 10.66.0.2 20-23 2>&1 | grep succeeded
printf 'USER admin\n' | nc -w 3 10.66.0.2 21
printf 'admin\n' | nc -w 3 10.66.0.3 23
```

8. Summarise the deception alerts

Pull every decoy signal into one view, the way a SIEM dashboard would. Each line is a lead worth investigating.

```
sleep 1
echo "== Honeypot hits by port and source =="
grep HONEYPOT /var/log/honeypot.log | awk '{print $3, $4}' | sed 's/:[0-9]*$//' | sort |
uniq -c
echo "== Honeyfile opens ==";  grep -c HONEYFILE /var/log/honeyfile-alert.log
echo "== Honeyfile readers (auditd) ==="; ausearch -k honeyfile -i 2>/dev/null | grep -o '
uid=[a-z0-9]*' | sort | uniq -c
echo "== Honeytoken ==="; /usr/local/bin/honeytoken-check.sh | head -1
```

Test it

```
grep -c "port=2222" /var/log/honeypot.log
grep -cE "port=(21|23) " /var/log/honeypot.log
ausearch -k honeyfile -i | grep -c "uid=intruder"
/usr/local/bin/honeytoken-check.sh | head -1
```

Expected: at least 1 hit on port 2222, at least 3 hits on ports 21/23 (scan plus probes), at least 1 auditd record for `uid=intruder`, and `ALERT: honeytoken AKIAHONEYTOKEN0LAB03 was used`.

Troubleshooting

Symptom	Fix
`nc` prints nothing and the log is empty	Check the decoy is listening with `ss -tlnp | grep 2222` (or `ip netns exec honeynet ss -tlnp`). If not, rerun the `nohup python3 ...` line; a port already in use stops the script.

Symptom	Fix
`RTNETLINK answers: File exists` when building the honeynet	The veth pair already exists from an earlier run. The step is safe to repeat; check with `ip -br addr show veth-host`.
`ausearch` shows `<no matches>`	Confirm the watch with `auditctl -l`, then read the file again as `intruder`. If auditd is not running, use the inotify alert in step 5 as your evidence.

Challenge

Turn the honeytoken check into a scheduled detective control: write a systemd timer (or cron entry) that runs `honeytoken-check.sh` every minute and logs alerts with `logger -t honeytoken`. Then plant a **database honeytoken**: a fake customer row in a SQLite table whose email address, if it ever receives mail or appears in a log, proves the table was exfiltrated.

Reflection

Why does a honeypot produce far fewer false positives than an IDS watching production traffic, and what is the risk of deploying a high-interaction honeypot on the production network?

Exam tips

- **Honeypot** = decoy host or service; **honeynet** = network of decoys; **honeyfile** = decoy file; **honeytoken** = decoy data or credential.
- Decoys are **detective** controls: they gather attacker tactics and tools and raise high-confidence alerts.
- Any interaction with a decoy is suspicious because it has no legitimate business use.
- A honeytoken can live anywhere data travels: code repositories, databases, documents and cloud credential files.

References

- MITRE Engage (adversary engagement and deception) — https://engage.mitre.org/
- MITRE D3FEND decoy techniques — https://d3fend.mitre.org/
- Thinkst Canarytokens (free honeytokens) — https://canarytokens.org/
- inotifywait man page — https://man7.org/linux/man-pages/man1/inotifywait.1.html

---